

SENDA: vehicle routing with single-trajectory search, explained and measured honestly

An educational guide to the LATTIMEX routing core: how it works, how it was evaluated and how to try it on your computer

Erick Martínez

LATTIMEX · independent research · Querétaro, Mexico
contacto@lattimex.com · <https://lattimex.com>

September 30, 2026

English translation of the Spanish original

Abstract

In a nutshell. Assigning deliveries to vehicles and deciding the order of visits is a classic optimization problem that cannot be solved exactly at large scale: heuristics are used instead, programs that find very good solutions in little time. This guide opens one of them, the routing core of the LATTIMEX engine, piece by piece, and shows how to measure its quality without fooling yourself. At the end, the reader can install the engine, which is open source (MIT), and check the results on their own computer.

This document explains, from scratch and with worked examples, how SENDA works and how it was evaluated. SENDA is a solver for the capacitated vehicle routing problem (CVRP). The solver follows a single search trajectory: it builds an initial solution, improves it with granular local search, restructures it with an adaptive large neighborhood search (ALNS), refines it with segment moves and SWAP*, and splits its time among two or three variants chosen from the features of the instance (a portfolio of configurations). For each component the guide explains what it is, why it matters and how much it contributes. A cascade experiment measures the quality available after each stage: on A/B/E the mean gap goes from 81.008 % after construction to 0.115 % with the full portfolio, and on X from 25.786 % to 4.121 %. With a frozen contract, 10 external replicates, the same clock for all methods and independent verification, SENDA obtains on 60 A/B/E instances a gap of 0.111 % versus 0.004 % for HGS-CVRP (it ties on 37 and HGS is better on 23). On 41 X instances it obtains 4.119 %, versus 1.450 % for HGS and 0.758 % for FILO2. The document includes a case study on a verification error and its correction.

Keywords: vehicle routing; CVRP; ALNS; granular search; SWAP*; reproducible evaluation; teaching optimization.

Contents

How to read this document	3
1 Introduction	4
2 The CVRP in ten minutes	4
2.1 Definition	4
2.2 How quality is measured: BKS and gap	5
2.3 The benchmarks	5
3 What SENDA is: one search trajectory per run	5

4	Map of the solver: how a solution flows	6
5	The components in detail	7
5.1	Initial construction	7
5.2	Granular local search	7
5.3	ALNS with simulated annealing	8
5.4	Segment moves and Or-opt-3	9
5.5	SWAP*	10
5.6	Intensification of the global best	11
5.7	FAST selector	11
5.8	The portfolio: the evaluated configuration	11
5.9	Parameters	12
6	How quality accumulates: the cascade experiment	12
7	Computational complexity: how much each step costs	14
7.1	Big- O notation	14
7.2	Why the CVRP is not solved exactly	15
7.3	Notation for the analysis	15
7.4	Cost of each component	16
7.5	Experimental check	17
7.6	Where the cost could go down	18
8	How to evaluate a heuristic without fooling yourself	18
9	Confirmatory results	19
9.1	A/B/E: almost optimal, but HGS is systematically better	19
9.2	X: the lightweight architecture stops being competitive	20
9.3	Ablation: what sustains the quality	20
9.4	Prospective holdout	21
10	Case study: an audit that corrected our own results	21
10.1	What was published	21
10.2	How the problem was detected	21
10.3	The cause	22
10.4	The correction	22
11	Discussion	22
12	Conclusions	23
13	Try it on your computer	23
A	Glossary	25
B	Literal rule of the FAST selector	26
C	Errata with respect to version 2	27

How to read this document

The text is aimed at students, logistics engineers and anyone who wants to understand a modern routing heuristic and the right way to measure its quality. No prior experience with metaheuristics is required: basic notions of graphs, probability and statistics are enough.

- Sections 2 and 3 present the problem and what SENDA is.
- Sections 4–6 are the teaching core: they describe each component, how the solution passes from one to the next and how much quality each step adds.
- Section 7 introduces big- O notation and estimates how much each component costs.
- Section 8 explains how to evaluate a heuristic without fooling yourself.
- Sections 9 and 10 present the results and a case study on a real error.
- Section 13 explains how to install the engine and repeat the measurements on your computer.

What you will learn. (1) What the CVRP is and why it is solved with heuristics; (2) how a routing solution is built, improved and diversified, with an example for each component; (3) how to estimate with big- O notation how much each component costs and why that limits quality under a fixed clock; (4) how to design a fair comparison (same clock, replicates, independent verification, paired statistics); (5) how a verification error can manufacture “wins” and how to detect it; and (6) how to go from an academic solver to an engine that plans real routes with three-dimensional loading.

The boxes serve four purposes:

Key idea

Summarizes the concept that should not be forgotten.

Worked example

Solves a small calculation, step by step and with concrete numbers.

Methodological lesson

Describes a possible error, or one that actually happened in this project, and how to avoid it.

To reproduce

Gives the SHA-256 fingerprints of the artifacts behind a figure and how to repeat the measurement.

About this edition. This guide accompanies the public LATTIMEX repository. The programs used for comparison (HGS-CVRP, FILO2 and the campaign scripts) are not part of the repository: they are described here so that the comparison is transparent, and Section 13 explains how to repeat it with copies obtained from their official sources. Raw data, contracts and logs are kept in the author’s archive; each figure cites the SHA-256 fingerprint of the artifact behind it. This edition includes the correction of the A/B/E comparison (Section 10). It is a translation of the Spanish original (`guia_senda.pdf`); in case of discrepancy, the original prevails. Instance names, configuration names and identifiers in the code are kept as they appear in the original.

1 Introduction

Every day, parcel, distribution and service companies decide which vehicle visits which customers and in what order. That decision, repeated thousands of times, is the *capacitated vehicle routing problem* (CVRP), first formulated by Dantzig and Ramser [5]. Its statement is short, but the number of possible solutions grows explosively with the number of customers. That is why, in practice, it is solved with *heuristics*: algorithms that find very good solutions in seconds, even though they cannot guarantee that they are optimal.

The state of the art in heuristics is dominated by hybrid genetic search HGS-CVRP [16, 15] and by very fast solvers such as FILO2 [1]. This work does not aim to beat them. Its goal is to build a *compact and auditable* solver—a single search trajectory, with no population of solutions and no crossover—and to answer three questions with reproducible evidence:

- Q1.** What does each component of the solver do, and how much quality does it add?
- Q2.** What final quality does it reach, with the same time as the reference solvers, on small (A/B/E) and large (X) instances?
- Q3.** What is learned when the evaluation protocol itself contains an error?

Contributions. (1) A complete description of SENDA, component by component, with pseudocode and numerical examples. (2) A measurement of the quality each stage adds, complementary to the classic ablation. (3) An evaluation protocol with a frozen contract, replicates, independent verification and per-instance inference. (4) An honest comparison against HGS and FILO2 that delimits where the lightweight architecture is competitive and where it is not. (5) A case study on a verification error and its correction.

2 The CVRP in ten minutes

2.1 Definition

A CVRP instance has a depot, identified as node 0; a set of customers $V = \{1, \dots, n\}$, each with a demand $q_i > 0$; a vehicle capacity Q ; and a distance $d(i, j)$ between every pair of points. A *route* leaves the depot, visits a sequence of customers and returns to the depot. A *solution* is a set of routes that satisfies three conditions:

- (i) every customer is visited exactly once (*coverage*);
- (ii) the total demand of each route does not exceed Q (*capacity*);
- (iii) the number of routes does not exceed the available fleet k , when the instance fixes it (*fleet*).

The objective is to minimize the total distance traveled:

$$C(S) = \sum_{r \in S} \sum_{(i,j) \in r} d(i, j). \quad (1)$$

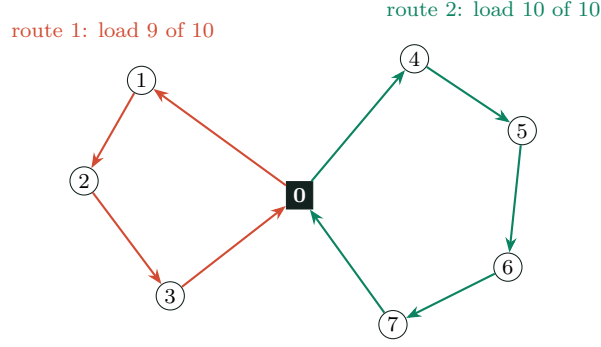


Figure 1: Solution of a small CVRP with 7 customers, capacity $Q = 10$ and fleet $k = 2$. Every customer is visited only once and no route exceeds the capacity.

2.2 How quality is measured: BKS and gap

For the CVRPLIB reference instances [4] the *best known solution* (BKS) is known; many of them are certified as optimal [10]. The quality of a heuristic is measured with the *gap*, the relative difference between its cost and the BKS:

$$\text{gap}(S) = 100 \cdot \frac{C(S) - C_{\text{BKS}}}{C_{\text{BKS}}} (\%). \quad (2)$$

Worked example: computing a gap

On instance B-n51-k7 the BKS is 1032 and uses $k = 7$ vehicles. If a solver delivers a solution of cost 1040 with 7 routes, its gap is $100 \cdot (1040 - 1032)/1032 = 0.78\%$. If it delivers one of cost 1032, its gap is 0%: it reached the BKS.

Key idea: a negative gap is an alarm

If the BKS is certified as optimal, *no* correct solver can obtain a negative gap. A negative gap means that the solution violates some constraint—almost always the fleet constraint—or that the verification is wrong. Section 10 shows that this happened in this project.

2.3 The benchmarks

- **A, B and E** (Augerat and Christofides [2, 3]): 63 classic instances with 12 to 100 customers and a fixed fleet. The name gives the size and the fleet: A-n32-k5 has 32 nodes (31 customers and the depot) and 5 vehicles.
- **X** (Uchoa et al. [14]): large and more varied instances. 41 of them are used here, with 199 to 428 customers and a free fleet. In X, the number k in the name is a reference and not a constraint.

3 What SENDA is: one search trajectory per run

SENDA (*Single-trajectory ENgine with Destroy-and-repair Adaptive search*; in Spanish, *senda* means “path”) is the routing core of the LATTIMEX engine. Its design trait is the one that gives it its name: it follows **a single search trajectory per run**. Within each run there is at all times one current solution—with all its routes—that is partially destroyed, repaired and improved; there is no population of solutions crossed with one another, as in the state-of-the-art genetic algorithms for the CVRP (for example, Vidal’s HGS-CVRP [15]).

The decision has practical advantages that matter in a production engine: the path of the solution can be followed step by step and explained (Section 4); effort is split among a few stages

with an explicit clock; and parallelism is obtained with a portfolio of independent trajectories. The final choice among its candidates is deterministic; since the budget of each run is wall-clock time and depends on the number of threads, the candidates —and therefore the answer— may vary between computers. The cost is also clear, and this guide measures it: on the pure CVRP, with the same clock, a well-tuned population algorithm is still more accurate (Section 9).

SENDA combines published ideas —ALNS [11], granular neighborhoods [13], penalized infeasibility and the SWAP* neighborhood [15]— in an original implementation. The contribution of this implementation lies in how they are put together: the configuration selector driven by instance features, the portfolio of arms with a shared clock and, in the LATTIMEX engine, the integration with three-dimensional load placement, fixed-fleet mediation and the unloading plan by door.

4 Map of the solver: how a solution flows

Before studying each component it helps to see the whole path. A single object circulates in SENDA: a *solution*, which is a list of routes together with its cost, the load of each route and a flag that says whether it is feasible. Each component receives a solution and delivers another, usually shorter; some may pass through infeasible solutions while they work, but what they deliver at the end is feasible. Figure 2 shows that path.

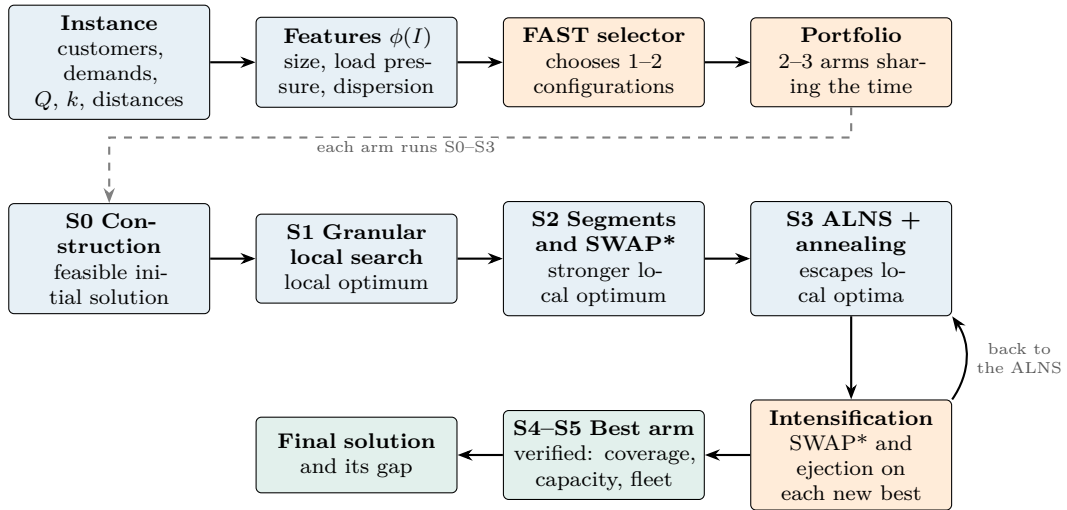


Figure 2: Flow of information in SENDA. The features of the instance decide how many arms run and with which configuration. Each arm goes through the chain S0–S3 with its share of the clock; intensification fires every time the ALNS finds a new global best. At the end each arm is verified and the best one is kept.

The order is not arbitrary. Each stage fixes a kind of defect that the previous one cannot see:

1. **Construction** quickly produces feasible routes, but with obvious crossings and detours.
2. **Granular local search** removes those defects with small moves until no small change improves: the solution sits at a *local optimum*.
3. **Segments and SWAP*** widen what counts as a “small move” and allow escaping local optima that the basic moves cannot break.
4. The **ALNS** destroys and rebuilds large parts of the solution, which lets it change its structure: which customers share a route. That is how it escapes deep local optima.
5. **Intensification** thoroughly polishes each new best solution.

6. The **portfolio arms** repeat the process with different configurations and keep the best result, which reduces the risk of a single trajectory stagnating.

5 The components in detail

Each component is presented with the same questions: what it is, how it works, an example, why it matters and how much it contributes. The contribution figures come from the ablation of Section 9.3 (what is lost when it is removed) and from the cascade of Section 6 (what is gained when it is added).

5.1 Initial construction

What it is. A fast procedure that produces a first complete solution.

How it works. If the fleet is fixed, *best-fit-decreasing* (BFD) is used: customers are sorted from largest to smallest demand and each one is assigned to the vehicle that, after loading it, would have the least free space left. If it fits in no vehicle, it is assigned to the least loaded one and the solution is temporarily infeasible; later stages repair it. Within each vehicle, the customer is inserted at the position that lengthens the route the least. If the fleet is free, *nearest neighbor* is used: from the depot, the closest customer that still fits is visited and, when the vehicle is full, a new route is opened.

Worked example: best-fit-decreasing

Let $Q = 10$, $k = 2$ and demands 7, 5, 4, 3, 1. The customer with demand 7 goes to vehicle A (3 left free). The one with 5 does not fit in A and goes to B (5 left). The one with 4 fits only in B (1 left). The one with 3 fits in A, which becomes full, and the one with 1 fits in B, which becomes full. Result: $A = \{7, 3\}$ and $B = \{5, 4, 1\}$, both with load 10.

Why it matters. With a fixed fleet, the first thing is to split the demand among the k vehicles without exceeding their capacity, and in many A/B/E instances the total demand almost fills the fleet. Placing the large customers first, while there is still room, increases the probability of a feasible split. Construction does not try to make routes short: the following stages take care of that.

How much it contributes. It is the starting point (stage S0 of the cascade): on average it leaves a gap of 81.008 % on A/B/E and 25.786 % on X.

5.2 Granular local search

What it is. A procedure that tries small changes to the solution and applies those that reduce the distance, until none improves it.

How it works. Five families of moves are used: *relocate* (move a customer to another position), *swap* (exchange two customers), *2-opt* (reverse a stretch of a route to remove a crossing), *2-opt** (exchange the tails of two routes) and *ejection chains* (move a customer that in turn displaces another). Trying all pairs of customers costs $O(n^2)$ per pass, so the *granular restriction* of Toth and Vigo [13] is applied: each customer only considers its $k = 15$ nearest neighbors, because it is almost never worth joining two very distant customers in one route. In addition, each move is evaluated with a constant-time cost formula, without rebuilding the route. To move customer u ,

whose neighbors in its route are p_u and n_u , between nodes x and y of another route:

$$\Delta = \underbrace{d(x, u) + d(u, y) - d(x, y)}_{\text{cost of inserting it}} - \underbrace{[d(p_u, u) + d(u, n_u) - d(p_u, n_u)]}_{\text{saving from removing it}}. \quad (3)$$

If $\Delta < 0$ and the target route has capacity, the move is applied.

Worked example: the delta of a relocate

Let $d(p_u, u) = 4$, $d(u, n_u) = 5$, $d(p_u, n_u) = 7$, $d(x, u) = 3$, $d(u, y) = 2$ and $d(x, y) = 4$. Removing u saves $4 + 5 - 7 = 2$ and inserting it between x and y costs $3 + 2 - 4 = 1$. So $\Delta = 1 - 2 = -1$: the move shortens the solution by one unit. The calculation needed six lookups in the distance matrix, without traversing any route.

Why it matters. It is the tool that does the fine work and the one that runs most often: stage S1, every ALNS candidate and intensification use it. The granular restriction and constant-time evaluation let it review many more moves in the same time than a search over all pairs of customers.

Capacity during polishing. The initial polish works with a *penalized* cost, $C(S) + \lambda \cdot \text{excess}(S)$: it tolerates capacity excesses if they save enough distance. If some excess remains at the end, it repeats the search with a penalty ten times larger to recover feasibility. That second search does not always succeed: when the load is very tight, the polish may end with a small excess (Section 6), and feasibility is recovered later, inside the ALNS.

How much it contributes. Removing the granular restriction—that is, using the full neighborhood—worsens the gap on X by 0.267 pp: with the same clock, the full neighborhood evaluates fewer useful iterations.

5.3 ALNS with simulated annealing

What it is. ALNS (*Adaptive Large Neighborhood Search* [12, 11]) is the main engine. In each iteration it *destroys* part of the solution—it removes between 10 and 35 % of the customers—and *repairs* it by reinserting them.

Why it is needed. Local search stops at a local optimum: no small modification improves. But the best solution may require a large change, for example ten customers changing route at once. The ALNS causes those large changes in a controlled way.

How it destroys. There are four operators, each with a different intuition:

- *random*: removes customers at random to diversify;
- *worst contribution*: removes, with a random bias, the customers that lengthen their route the most, that is, those that are probably badly placed;
- *Shaw* [12]: removes a group of customers close to one another, so that they are redistributed together;
- *route removal*: removes whole routes. It is the operator meant to reduce the number of routes; that is why, when the solution exceeds the fleet, it is chosen with probability 0.8.

How it repairs. *Greedy* repair inserts each customer at its cheapest position. *Regret-2* repair first inserts the customer that would lose the most if left for later.

Worked example: regret-2 repair

Customers a and b remain to be inserted. The best insertion of a costs 5 and the second best 12, so its “regret” is $12 - 5 = 7$. The best for b costs 3 and the second 4: regret 1. Although b is cheaper, regret-2 inserts a first: if another customer takes its best place, a would have to pay 12, while b would only lose one unit.

Capacity during repair. In the standard arms, repair assigns a huge penalty (10^9) to any capacity excess, so in practice it only inserts where the customer fits. The penalized arm of the portfolio (Section 5.8) relaxes this rule.

How it learns. Each operator has a weight. It is chosen by roulette, with probability proportional to its weight, and then updated as $w \leftarrow 0.99w + 0.01\sigma$, with reward $\sigma = 5$ if it produced a new global best, 2 if it improved the current solution, 0.5 if it was only accepted and 0 if it was rejected. This way, operators that work on the current instance are used more.

How it decides whether to accept. The repaired candidate is polished with local search and accepted according to simulated annealing [8]: if it improves, it is always accepted; if it worsens by Δ , it is accepted with probability $e^{-\Delta/T}$. The temperature decreases geometrically, $T(f) = T_0(T_f/T_0)^f$, where $f \in [0, 1]$ is the fraction of the budget used, $T_0 = 0.02C$ and $T_f = 0.0002C$, with C the cost of the best initial solution.

Worked example: accepting a worse solution

If the candidate is 10 units longer ($\Delta = 10$) and the temperature is $T = 20$, it is accepted with probability $e^{-10/20} = 0.61$. At the end of the search, with $T = 1$, the same decision would have probability $e^{-10} \approx 0.00005$. The algorithm explores a lot at the beginning and becomes demanding at the end.

Algorithm 1 ALNS core of one SENDA arm

Input: solution S (already polished), budget B in seconds or iterations

Output: best feasible solution S^*

```
1:  $S^* \leftarrow S$ ; weights  $w \leftarrow \mathbf{1}$ 
2: while  $B$  is not exhausted and there have not been 5 000 consecutive iterations without improvement do
3:   choose destroy  $\delta$  and repair  $\rho$  by roulette with weights  $w$ 
4:    $S' \leftarrow \rho(\delta(S))$ ; polish  $S'$  with granular local search
5:   if  $S'$  is feasible and  $C(S') < C(S^*)$  then
6:     intensify  $S'$  (SWAP* and ejection);  $S^* \leftarrow S'$ ;  $S \leftarrow S'$ ;  $\sigma \leftarrow 5$ 
7:   else if  $\Delta = C(S') - C(S) < 0$  or  $u \sim U(0, 1) < e^{-\Delta/T(f)}$  then
8:      $S \leftarrow S'$ ;  $\sigma \leftarrow 2$  if  $\Delta < 0$ , otherwise 0.5
9:   else
10:     $\sigma \leftarrow 0$ 
11:   end if
12:    $w_\delta \leftarrow 0.99w_\delta + 0.01\sigma$ ;  $w_\rho \leftarrow 0.99w_\rho + 0.01\sigma$ 
13: end while
14: return  $S^*$ 
```

Why it matters and how much it contributes. It is the dominant component. In the ablation, removing the ALNS—leaving only construction and polishing—raises the mean gap on X from 4.285 % to 16.702 %: a loss of 12.4 points. In the cascade (Section 6), the ALNS is the

first stage that always delivers feasible solutions, and on A/B/E it lowers the gap of the feasible solutions from 3.475 % to 0.319 %.

5.4 Segment moves and Or-opt-3

What it is. An extension of local search: instead of moving one customer, a *segment* of two or three consecutive customers is moved (*relocate-2*, *Or-opt-3*), or a pair of customers is exchanged for a single one (*swap-2*).

Why it matters. Often two consecutive customers are well placed next to each other, but the whole pair would be better in another route. Moving them separately takes two steps, and the first one almost always worsens the solution, so local search does not take it. Moving them together turns that improvement into a single step that is detected.

How much it contributes. Removing them raises the gap on X by 0.528 pp; on 36 of 41 instances the full version is better. It is the second most important component after the ALNS.

5.5 SWAP*

What it is. A move between two routes proposed by Vidal for HGS-CVRP [15]. The classic *swap* exchanges customer u of route A with customer v of route B, and each takes the exact position of the other. SWAP* also exchanges them, but places each one at its *best* position in the other route.

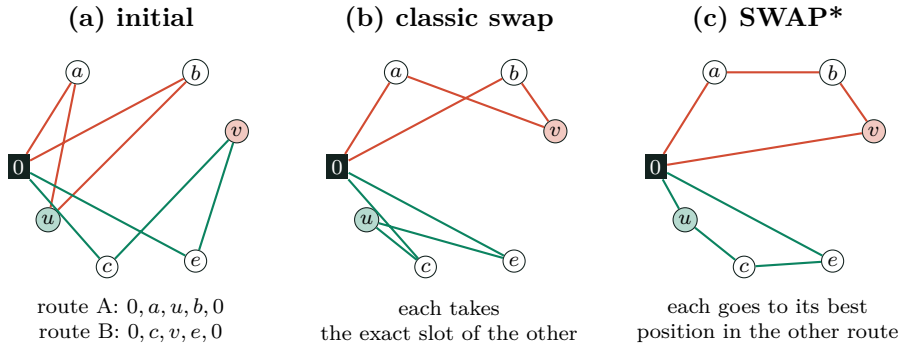


Figure 3: Difference between the classic swap and SWAP*. The points do not move; only the routes change. In (a), u is badly placed in the red route and v in the green one. The classic swap (b) exchanges them, but each ends up in the slot the other left, far from where it should be. SWAP* (c) exchanges them and places each at its best position, so it finds improvements that the classic swap discards.

How it works. The cost change is

$$\Delta_{\text{SWAP}^*}(u, v) = \text{ins}_{B \setminus v}^*(u) + \text{ins}_{A \setminus u}^*(v) - \text{rem}_A(u) - \text{rem}_B(v), \quad (4)$$

where rem is what is saved by removing the customer from its route and ins^* the cost of its best insertion in the other route. To make it fast, the three best insertion positions of each customer in each route are stored in advance; this way, the best place is obtained in constant time even if the outgoing customer occupies one of them. The original version chooses which pairs of routes to compare using polar sectors, which requires coordinates; ours uses the granular graph and also works with distance matrices without coordinates.

Worked example: why SWAP* finds more

Removing u from A saves 10 and removing v from B saves 9. In the classic swap, u in the slot of v costs 12 and v in the slot of u costs 11: $\Delta = 12 + 11 - 10 - 9 = +4$, so it is rejected. In SWAP*, the best position of u in B costs 7 and that of v in A costs 8: $\Delta = 7 + 8 - 10 - 9 = -4$, and the exchange improves the solution by 4 units.

Why it matters and how much it contributes. It explores much richer exchanges between routes at the same cost per evaluation. Removing it raises the gap on X by 0.413 pp, with 38 of 41 instances in favor of the full version. It is applied in the initial polish and in intensification, but not to every ALNS candidate, because that would reduce the number of iterations too much.

5.6 Intensification of the global best

Every time the ALNS finds a new best feasible solution, it is polished thoroughly: granular local search, up to two passes of SWAP* and ejection chains, with a maximum of 50 polishes per arm. This is when SWAP* and ejection chains do their work: they are reserved for solutions that have already proven promising.

5.7 FAST selector

What it is. A frozen rule that, from features that are cheap to compute, decides which additional configuration to run besides the base one.

How it works. It computes the number of customers n , the fleet k , the load pressure $\sum q_i / (kQ)$, the coefficients of variation of the demand and of the distance to the depot, the angular concentration of the customers around the depot and their density. With those values it walks a list of conditions (Appendix B) and the first one that holds chooses one of the configurations in Table 1: wider neighborhoods, more aggressive destruction or more iterations.

Table 1: Configurations the selector can choose. All use segments and SWAP* with two passes.

Configuration	Idea	neighbors k	destruction
baseline_cap2_calls50	base branch, always runs	15	10–35 %
granular_k25 / granular_k40	wider neighborhood	25 / 40	10–35 %
large_destroy_k25	more aggressive destruction	25	15–45 %
medium_destroy_k25	moderate destruction	25	10–25 %
long_10k_k25	up to 10 000 iterations	25	10–35 %

Why it matters and how much it contributes. There is no ideal configuration for every instance: with very variable demand it pays to destroy more; with concentrated customers, to widen the neighborhood. Replacing the chosen branch with a restart of the base branch raises the gap on X by 0.145 pp.

5.8 The portfolio: the evaluated configuration

What it is. It is the frozen configuration of SENDA evaluated on A/B/E. Its operational definition:

1. one arm with the base configuration and another with the configuration chosen by FAST, if different;
2. Or-opt-3 active and a stagnation cut: an arm stops after 5 000 iterations without improvement;

3. an additional **penalized arm** with the last configuration.

Each arm receives an equal share of the clock (80 % of the budget divided among 2 or 3 arms) and the best verified result is kept. On X the FAST-PEN4 variant was used, which differs in that its standard arms use neither Or-opt-3 nor the stagnation cut.

The penalized arm. In the standard arms, the ALNS repair treats capacity as a hard constraint: it never accepts an excess. The penalized arm allows passing through solutions with capacity excess, paying a proportional penalty:

$$C_\lambda(S) = C(S) + \lambda \cdot \sum_{r \in S} \max\left(0, \sum_{i \in r} q_i - Q\right). \quad (5)$$

The weight λ adjusts automatically every 50 iterations: if fewer than 25 % of the candidates are feasible, λ grows by 25 %; if more than 45 % are, it decreases by 15 %. This keeps the search near the frontier between feasible and infeasible solutions, which is where the best ones usually are.

Key idea: why penalization helps

When capacity is very tight, the feasible solutions form separate islands: to go from a good one to a better one you have to cross solutions with excess. The penalty builds that bridge. Since only the best *feasible* solution of each arm counts, the penalized arm never delivers an invalid solution. Its cost is something else: it uses its share of the clock, which in the worst case contributes nothing.

How much it contributes. Replacing the penalized arm with a normal restart of the same time raises the gap on X by 0.145 pp (23 instances in favor and 18 against; $p_H = 0.048$, the weakest evidence of the ablation). In a comparison at equal computation, adding it reduces the gap on X from 3.678 % to 3.461 % (31 of 41 instances, $p = 4.3 \times 10^{-5}$).

5.9 Parameters

Table 2: Solver parameters. None is tuned per instance.

Component	Parameter	Value
Granular	neighbors per customer (all if $n \leq 60$)	15
ALNS	destroyed fraction	$U(0.10, 0.35)$ depending on branch
ALNS	weight decay; rewards	0.99; 5 / 2 / 0.5 / 0
Annealing	T_0 , T_f (fraction of the cost)	0.02, 0.0002
Fleet	probability of forcing route removal	0.8
Intensification	SWAP* passes; polishes per arm	2; 50
Portfolio	stagnation cut; clock reserve	5 000 iterations; 20 %
Penalized arm	λ adjustment every 50 iterations	$\times 1.25$ / $\times 0.85$

6 How quality accumulates: the cascade experiment

The ablation answers “how much is lost if I remove a component?”. The cascade answers the complementary question: “what quality do I have after each stage?”. Six stages were run with the same 10 seeds of the protocol, on the 63 A/B/E instances (fixed fleet) and the 41 X instances (free fleet). The component-ablation binary was used, because it is the only one that allows stopping the solver before the ALNS; its full version obtained a gap of 4.285 % on X, close to but not identical to the 4.119 % of the confirmatory binary. The stages are:

Stage	Solution available
S0	construction, with no improvement
S1	construction + basic granular local search
S2	construction + local search with segments, Or-opt-3 and SWAP*
S3	first arm of the full portfolio (with ALNS), with its share of the clock
S4	best of S3 and the arm with the configuration chosen by FAST
S5	full portfolio: the penalized arm is added

S1 and S2 are two independent polishes of the same construction: S2 does not start from S1, but uses a richer set of moves. That is why, in a particular replicate, S2 may end up worse than S1, even though it is better on average. Likewise, each arm of S3–S5 does its own construction, polish and ALNS.

Three measures are reported for each stage, because a single one is misleading:

- the **percentage of feasible solutions** the stage delivers;
- the **gap of the feasible solutions**, averaged first per instance (in parentheses, how many instances had at least one feasible replicate);
- the **available gap**, the rule fixed before seeing the results: if the stage delivers an infeasible solution, the solution actually in hand is still the construction.

Table 3: Quality after each stage (10 replicates per instance). “Feas.” is the percentage of feasible solutions; “feasible gap” averages only the feasible solutions (in parentheses, the number of instances with at least one); “available gap” replaces each infeasible solution with the construction.

Stage	A/B/E (63 inst., 2s, fixed fleet)			X (41 inst., 4s)		
	feas.	feasible gap	available gap	feas.	feasible gap	available gap
S0 construction	100 %	81.008 % (63)	81.008 %	100 %	25.786 % (41)	25.786 %
S1 + granular local	83 %	4.671 % (62)	18.972 %	71 %	6.262 % (39)	14.453 %
S2 + segments and SWAP*	49 %	3.475 % (51)	46.181 %	42 %	5.244 % (32)	19.508 %
S3 + ALNS (base arm)	100 %	0.319 % (63)	0.319 %	100 %	4.971 % (41)	4.971 %
S4 + FAST branch	100 %	0.243 % (63)	0.243 %	100 %	4.459 % (41)	4.459 %
S5 + penalized arm	100 %	0.115 % (63)	0.115 %	100 %	4.121 % (41)	4.121 %

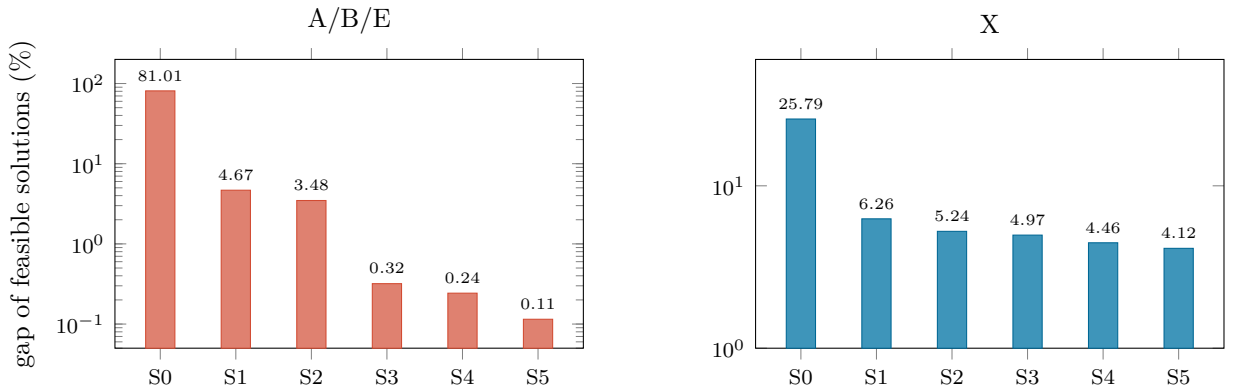


Figure 4: Gap of the feasible solutions after each stage (logarithmic scale). On A/B/E the decisive drop happens when the ALNS comes in (S3); on X local search already does much of the work and the extra arms (S4, S5) contribute more than on A/B/E.

Worked example: a solution in transit

On instance A-n80-k10 (BKS 1763), replicate 0, the cost evolves like this: construction 3180 (80.37 %); local search 1827 (3.63 %); segments and SWAP* 1865 (5.79 %); first arm with ALNS 1783 (1.13 %); with the FAST branch 1766 (0.17 %); and with the penalized arm 1766 (0.17 %).

Reading.

1. **Construction does not look for short routes.** Best-fit-decreasing splits the demand without looking at geography, which is why on A/B/E it leaves gaps close to 80 %. On X, nearest neighbor does look at distances and starts from a smaller gap.
2. **Local search makes the largest reduction in distance.** With the polish alone, the gap of the feasible solutions drops to 4.671 % on A/B/E and to 6.262 % on X.
3. **Segments and SWAP* shorten further, but break capacity.** The feasible solutions of S2 are better than those of S1, but only 49 % of the S2 polishes ended feasible on A/B/E (42 % on X). The penalized polish crosses into the infeasible region to save distance and does not always manage to come back. That is why the available gap of S2 is bad: in half of the cases there is nothing but the construction.
4. **The ALNS is the first stage that delivers both quality and feasibility in 100 % of the trials.** On A/B/E it lowers the gap to 0.319 %, with a median close to zero. Its repair and its local search treat capacity as a hard constraint at every iteration.
5. **The extra arms fine-tune.** On A/B/E, the FAST branch and the penalized arm reduce the gap from 0.319 % to 0.115 %, and the number of instances that reach the BKS in all their replicates goes from 31 to 37. On X, where a single arm has little time, their relative contribution is larger: from 4.971 % to 4.121 %.

Methodological lesson: an invalid solution always looks better

If the cost of all S2 solutions were averaged, including the infeasible ones, its gap would be 0.238 % on A/B/E: *better* than that of the ALNS (0.319 %). It is the same trap as the case study of Section 10: violating a constraint —capacity or fleet— almost always allows shorter routes. That is why every comparison must count only verified solutions.

Methodological lesson: cascade and ablation do not say the same thing

In the cascade, what is attributed to each stage depends on the order in which they are added: local search seems to do most of the work because it goes first and finds a solution with plenty of room for improvement. In the ablation, removing a component measures what the others *cannot* compensate: without the ALNS, the gap on X rises 12.4 points, because local search alone does not reach the solutions the ALNS does find. Both views are needed. For example, the cascade does not show the value of the granular restriction, because it is not a stage but the way all of them run; only the ablation reveals it.

7 Computational complexity: how much each step costs

The previous sections explained what each component does and how much quality it contributes. An equally important question remains: *how much does it cost*. In SENDA the budget is a fixed clock, so the cost of each step decides how many iterations fit in that clock and, with that, how far the search gets. This section introduces the tool to reason about that cost, big- O notation, applies it to each component and checks the prediction with a measurement.

7.1 Big- O notation

Counting seconds does not work for comparing algorithms: the same program takes a different time on each computer. What is stable is *how* the number of operations *grows* when the input size n grows. Big- O notation describes that growth. An algorithm is said to be $O(g(n))$ if, beyond a certain size, its number of operations never exceeds a fixed multiple of $g(n)$. Formally, $f(n) = O(g(n))$ if there exist constants $c > 0$ and n_0 such that

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0. \quad (6)$$

In practice three rules are enough:

- **Constants and lower-order terms are dropped.** $3n^2 + 50n + 7$ is $O(n^2)$: for large n , the n^2 term dominates the others.
- **Steps in sequence add up and the largest dominates.** Sorting ($O(n \log n)$) and then walking the list ($O(n)$) costs $O(n \log n)$.
- **Nested loops multiply.** Visiting each customer and, for each one, all the others costs $O(n \cdot n) = O(n^2)$.

Table 4: How the most common complexity functions grow. Logarithms are base 2.

Function	$n = 100$	$n = 1\,000$	Example in this guide
$O(1)$	1	1	evaluating the delta of a move (Eq. 3)
$O(\log n)$	6.6	10	—
$O(n)$	100	1 000	computing the cost of a solution
$O(n \log n)$	664	9 966	sorting customers by demand
$O(n^2)$	10^4	10^6	computing the distance matrix
$O(n^3)$	10^6	10^9	regret-2 repair with fixed-length routes
$O(2^n)$	1.3×10^{30}	1.1×10^{301}	trying every subset of customers
$O(n!)$	9.3×10^{157}	4.0×10^{2567}	trying every visiting order

Key idea: big- O describes growth, not time

Two $O(n)$ algorithms can take very different times: the notation hides the constants. What it does say is what happens as the instance grows. If a step is $O(n^2)$, doubling the number of customers multiplies its cost by about 4; if it is $O(n^3)$, by about 8.

7.2 Why the CVRP is not solved exactly

The CVRP is *NP-hard* [9]: it contains the traveling salesman problem as a special case (a single vehicle with enough capacity), and no algorithm is known that solves it exactly in polynomial time, that is, in $O(n^c)$ for some constant c . An exhaustive search would have to review, just for the visiting order, on the order of $n!$ possibilities, plus every way of splitting the customers among vehicles.

Worked example: the size of the search space

With a single vehicle and $n = 20$ customers there are $20! \approx 2.4 \times 10^{18}$ visiting orders. A computer that evaluated one billion orders per second would need $2.4 \times 10^{18}/10^9 \approx 2.4 \times 10^9$ seconds: about 77 years. With 21 customers, 21 times longer.

Modern exact methods certify optima on many reference instances [10], but their time grows in ways that are hard to predict. A heuristic trades the guarantee of optimality for something that can be controlled: each iteration costs a *polynomial* number of operations, and the number of iterations is set by the clock.

7.3 Notation for the analysis

Besides n (number of customers), the cost of SENDA depends on four quantities:

- k : neighbors per customer in the granular restriction (15, 25 or 40 depending on the branch; all other customers if $n \leq 60$);
- R : number of routes;
- $L \approx n/R$: average length of a route, in customers;
- q : customers removed by an ALNS destruction, $q = \theta n$ with θ between 0.10 and 0.35 (Table 2).

Routes are stored as arrays of customers. Inserting or removing a customer from an array requires shifting the ones that follow it, so it costs $O(L)$.

7.4 Cost of each component

Table 5 summarizes the analysis of the code in `native/send_core.cpp`. The figures are worst case per call.

Table 5: Complexity of each SENDA component.

Component	Cost	Where it comes from
<i>Once per run</i>		
Distance matrix	$O(n^2)$	one distance per pair of nodes; 4 bytes per pair (4 MB with 1 000 customers)
Granular neighbor lists	$O(n^2 \log n)$	for each customer the distances to the others are sorted
<i>Best-fit-decreasing</i> construction	$O(n \log n + nR + nL)$	sort; choose a vehicle; insert at the best position
Nearest-neighbor construction	$O(n^2)$	at each step the closest customer among those left is searched
<i>Local search</i>		
Evaluating a move	$O(1)$	six to eight lookups in the matrix (Eq. 3)
Applying a move	$O(L)$	insert or remove in the route array and update its loads
One granular pass	$O(nk)$ evaluations	each customer tries its k neighbors with each family of moves
<i>One ALNS iteration</i>		
Random destruction	$O(n)$	shuffle and remove
Worst-contribution destruction	$O(n \log n + qn)$	sort by contribution; each removal reshuffles the list
Shaw destruction	$O(qn \log n)$	before each removal the list is re-sorted by closeness
Route destruction	$O(n + R \log R)$	choose routes and separate their customers
Greedy repair	$O(q(n + k + R))$	per customer: reindex the solution and review its insertion options
Regret-2 repair	$O(qn + q^2(k + R) \log(k + R))$	each round recomputes and sorts the options of <i>all</i> pending customers
<i>Intensification</i>		
SWAP* (one pass)	$O(PL^2)$	$P \leq \min(R^2, nk)$ pairs of neighboring routes; in each pair, the three best insertions of each customer in the other route
Ejection chains (one pass)	$O(nRL^3) = O(n^2L^2)$	each customer against each full route, each pair of customers to eject and one reinsertion per combination
Removing a surplus route	$O(n^2)$	each candidate route reinserts its customers at every position of the others

Worked example: why the granular restriction changes the order of magnitude

With $n = 400$, a pass that tried every pair of customers would make about $400^2 = 160\,000$ evaluations per family of moves. With $k = 15$ it makes $400 \times 15 = 6\,000$: 27 times fewer. Since each evaluation is $O(1)$, the full pass goes from $O(n^2)$ to $O(nk)$, which is linear in n because k is fixed.

The cost of one ALNS iteration. An iteration adds up one destruction, one repair, the local search of the candidate and the computation of its cost ($O(n)$). Local search has a time cap (0.08 s per candidate), so it does not grow without bound. With $q = \theta n$, the dominant terms are the Shaw destruction, $O(n^2 \log n)$, and the regret-2 repair, $O(n^2(k + R) \log(k + R))$. If the route length is fixed, R grows with n and regret-2 approaches $O(n^3)$. The prediction is, then, that an iteration becomes between 4 and 8 times more expensive each time n doubles.

Why SWAP* and ejection chains are kept in reserve. One pass of ejection chains is $O(n^2 L^2)$: with 400 customers in routes of 12 there are on the order of a million combinations (incoming customer, target route and pair of ejected customers), each with reinsertions of cost $O(L)$, about 2×10^7 steps per pass. Applying it to every ALNS candidate would use up the clock in a few iterations. That is why SWAP* and the chains are only applied in the initial polish and to each new global best (Section 5.5): they are expensive operators used where they pay off the most.

7.5 Experimental check

To contrast the analysis with the real program, we measured how many ALNS iterations SENDA completes in a fixed clock, on random instances of 100 to 800 customers. Table 6 and Figure 5 show the result.

Table 6: ALNS iterations in 4 s (one thread, median of 3 seeds) and mean time per iteration.

n	Iterations	ms per iteration	Factor when doubling n
100	11 073	0.32	—
200	2 714	1.30	$\times 4.1$
400	703	5.01	$\times 3.9$
800	99	35.56	$\times 7.1$

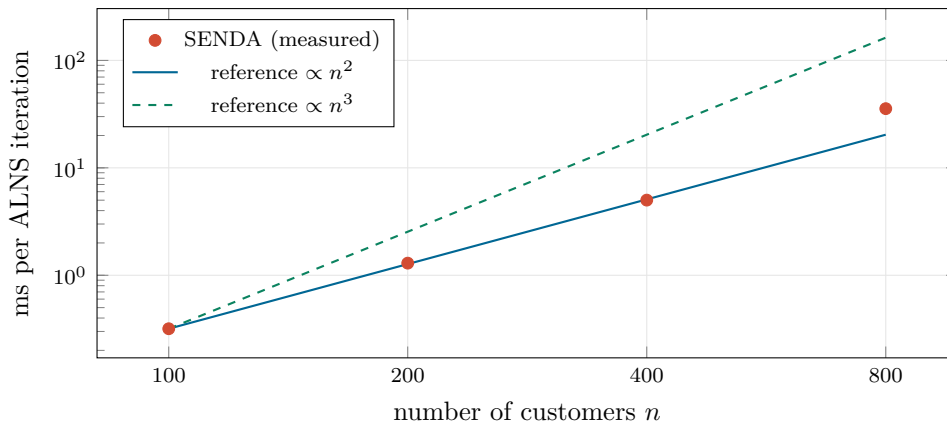


Figure 5: Time per iteration versus n on a logarithmic scale. On this scale, a function cn^a is a straight line with slope a . Up to 400 customers the points follow the n^2 reference; from 400 to 800 the slope approaches that of n^3 .

The measurement confirms the analysis. Between 100 and 400 customers, each doubling of n multiplies the cost of an iteration by about 4, as befits an $O(n^2)$ cost. From 400 to 800 the factor rises to 7.1, closer to n^3 : this is expected when the q^2R term of regret-2 starts to weigh, although in that range intensification also plays a role, because each polish of the global best costs more and there are fewer iterations to spread it over. The measurement does not separate the two effects.

Methodological lesson: with a fixed clock, complexity turns into quality

With a cost per iteration of order n^2 , the number of iterations that fit in the clock falls as $1/n^2$: from 100 to 400 customers, from 11 073 to 703 iterations, 16 times fewer. This is consistent with what was observed on the X instances (Section 9): with 199 to 428 customers and little more than one second per arm, the ALNS has on the order of hundreds of iterations and barely improves what local search already achieved. For a heuristic to scale it is not enough for each step to be polynomial; the exponent matters.

To reproduce

Binary compiled from `native/send_core.cpp` (SHA-256 `d63db4bc6436...`) with `python -m lattimex build`; direct call to `senda_solve` with a 4 s clock, free fleet, $k = 15$, segment moves and SWAP* active (`use_segment_moves=3`), no penalized arm and seeds 1, 2 and 3. Instances: depot and customers uniform in $[0, 1000]^2$, integer demands uniform between 1 and 10, capacity 66 (about 12 customers per route). Machine: Intel Core i5-8250U. Absolute figures change with the machine; the factors when doubling n are what should be reproduced.

7.6 Where the cost could go down

The analysis also points to possible improvements. None is applied in the engine; they are proposed as exercises.

1. **Shaw destruction without re-sorting.** Instead of sorting the whole list before each removal ($O(n \log n)$), choose among the granular neighbors of the reference customer, which are already sorted: $O(k)$ per removal.
2. **Incremental regret-2.** When a customer is inserted only the options that touch the modified route change. Keeping the two best options of each pending customer and recomputing only the affected ones avoids repeating the work for all pending customers in every round.
3. **Neighbors by partial selection.** To obtain the k nearest it is not necessary to sort all of them: `std::nth_element` separates them in $O(n)$ per customer, and building the lists drops from $O(n^2 \log n)$ to $O(n^2)$.

8 How to evaluate a heuristic without fooling yourself

A heuristic is random and its quality depends on the time it is given. That is why the way it is measured matters as much as the algorithm. This project followed six rules.

1. **The same clock for everyone.** The main profile gives each trial 2 s of wall-clock time on a single thread on A/B/E and 4 s on X, with a 10 % tolerance. A trial that exceeds the tolerance is discarded. SENDA reserves 20 % of the limit for reading the instance and verifying the solution, so its core uses the remaining 80 %.
2. **Replicates, not the best of ten.** Each instance is solved 10 times with different seeds and the *mean* is reported. Choosing the best of ten runs exaggerates the quality a user would get from a single execution.

3. Independent and uniform verification. Each solution is checked outside the solver: coverage, capacity, fleet and cost are recomputed. The rule must be *the same* for all methods (Section 10).

4. The unit is the instance. The replicates of each instance are averaged first, and then the methods are compared instance by instance with the Wilcoxon signed-rank test [17]. Bootstrap intervals [6] are added and, when there are several tests, the Holm correction [7].

Worked example: the intuition behind the Wilcoxon test

On five instances, the gap difference $A-B$ is $+0.30$, $+0.10$, -0.05 , $+0.20$ and 0 . The zero is discarded, the absolute values are sorted and given ranks: $0.05 \rightarrow 1$, $0.10 \rightarrow 2$, $0.20 \rightarrow 3$ and $0.30 \rightarrow 4$. The sum of the negative ranks is $W^- = 1$ and that of the positive ones $W^+ = 9$. If A and B were equally good, both sums would be similar; here B wins on almost all of them. With only five instances the p -value is large; with 60 instances and $W^- = 0$, as in our $A/B/E$ comparison, it is tiny.

5. Freeze before looking. Before running, a *contract* is frozen: a file with the code, the parameters, the instances, the seeds and the planned tests, identified by its SHA-256 fingerprint. Any later change requires a recorded *amendment*. In addition, the selector was validated on a *holdout*: 10 X instances that were not looked at during its design.

6. Do not claim equivalence without a margin. A median of differences equal to zero or a large p -value do not prove that two methods are equal. Without an equivalence margin declared in advance, only differences are reported.

9 Confirmatory results

9.1 A/B/E: almost optimal, but HGS is systematically better

Table 7: A/B/E with a verified fixed fleet (2s, 10 replicates). Gap and time are per-instance means. W/T/L counts the instances won, tied and lost by SENDA. HGS was run in the same block and on the same machine (Section 10).

Method	n	Mean gap	Median	Worst	Time (s)	W/T/L
SENDA (portfolio)	60	0.111 %	0.000 %	1.172 %	1.58	0/37/23
HGS-CVRP	60	0.004 %	0.000 %	0.094 %	2.02	reference

SENDA reaches the BKS on most instances and its mean gap is one tenth of a point. Even so, it wins none: it ties on 37 and HGS is better on 23 (Wilcoxon $W = 0$, $p = 2.4 \times 10^{-7}$). Figure 6 shows the difference on each instance and, in gray, the figures published in the previous version, whose three SENDA “wins” were solutions with one vehicle too many.

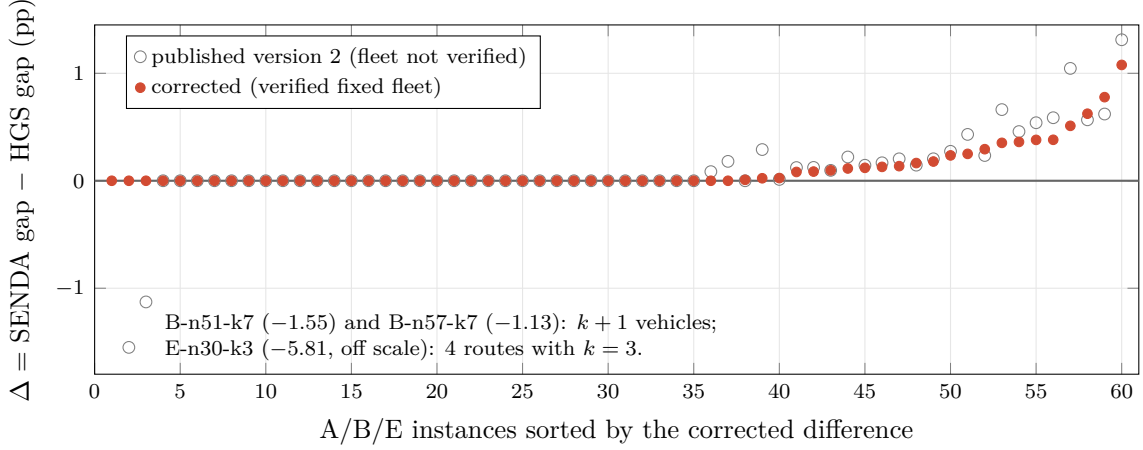


Figure 6: Gap difference per instance on A/B/E. The negative values of the published version (gray circles) disappear when the fleet is enforced: with k vehicles, SENDA ties with HGS on those three instances.

9.2 X: the lightweight architecture stops being competitive

Table 8: X instances (41, 4s, free fleet; protocol campaign v2).

Method	Mean gap	Median	Worst	Time (s)	W/T/L of SENDA
SENDA (X variant)	4.119 %	3.756 %	7.240 %	3.42	—
HGS-CVRP	1.450 %	1.452 %	3.101 %	4.17	0/0/41
FILO2	0.758 %	0.674 %	1.897 %	4.06	0/0/41

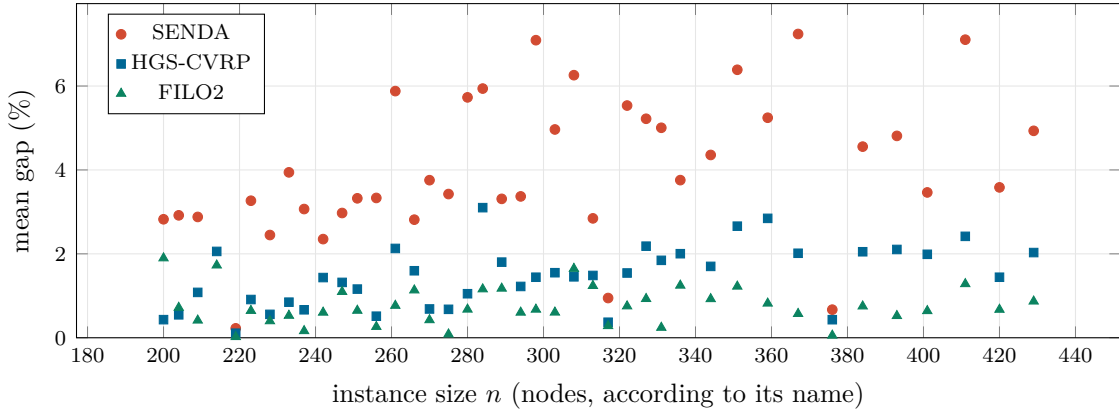


Figure 7: Gap per instance on X. HGS and FILO2 are better on all 41 instances ($p = 9.1 \times 10^{-13}$ in both comparisons). The gap is not due to a few outliers.

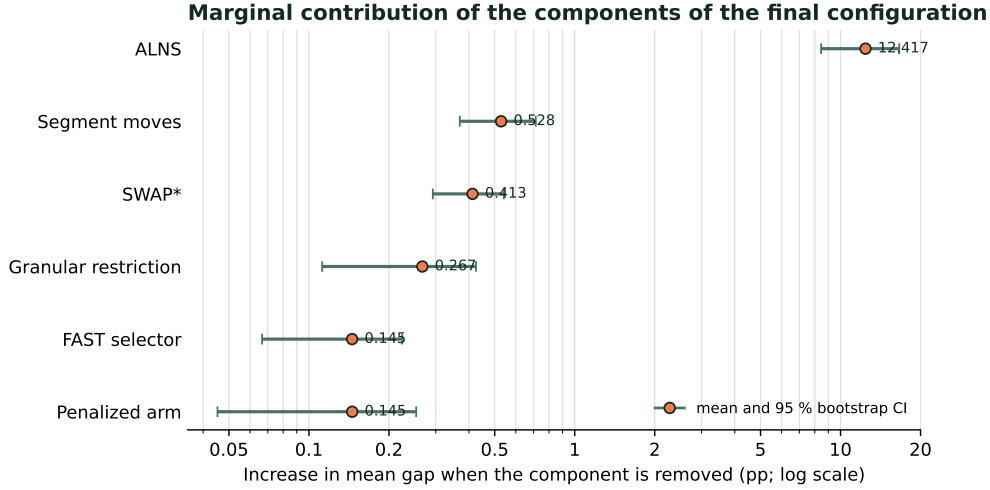
Time versus quality. In an earlier, non-confirmatory exploration with an iteration budget—about 18 times more wall-clock time than HGS—, SENDA reached 1.08 % on the 31 development X instances, versus 0.95 % for HGS. The gap on X is, above all, a matter of speed: the architecture needs much more time than HGS to reach similar quality. Section 7 shows why: the cost of an ALNS iteration grows at least as n^2 .

9.3 Ablation: what sustains the quality

To know which component matters, one is removed at a time with the same budget (41 X instances, 10 replicates, Holm correction over six comparisons).

Table 9: Ablation by component on X. Δ is the gap of the full version minus the gap without the component; a negative value favors the full version. p_H is the Holm-adjusted p -value.

Component removed	W/T/L	Gap without it	Δ (pp)	p_H
ALNS	40/1/0	16.702 %	-12.417	1.1×10^{-11}
Segment moves	36/0/5	4.813 %	-0.528	7.9×10^{-9}
SWAP*	38/0/3	4.698 %	-0.413	4.1×10^{-8}
Granular restriction	28/0/13	4.552 %	-0.267	5.7×10^{-3}
FAST selector	29/0/12	4.430 %	-0.145	2.5×10^{-3}
Penalized arm	23/0/18	4.430 %	-0.145	4.8×10^{-2}



Positive values mean deterioration when the component is removed; all six comparisons pass Holm.

Figure 8: Increase in the gap when each component is removed (logarithmic scale). All 95 % bootstrap intervals lie above zero.

Key idea: a hierarchy of mechanisms

The ALNS is the engine: without it, the gap multiplies by four. Moves between routes (segments and SWAP*) form a second level. The selector and the penalized arm are refinements of smaller magnitude.

9.4 Prospective holdout

The selector and its thresholds were frozen, with their SHA-256 fingerprint, before running 10 X instances that had never been observed (X-n351-k40 to X-n429-k61). On them the mean gaps were 4.799 % for SENDA, 1.998 % for HGS and 0.740 % for FILO2. The order between methods is preserved: the selector works as a rule outside the design instances, but it does not make SENDA the best solver on X.

10 Case study: an audit that corrected our own results

10.1 What was published

Version 2 reported on A/B/E a gap of 0.0120 % for SENDA versus 0.0059 % for HGS, with 3 instances won by SENDA, 33 ties and 25 losses.

10.2 How the problem was detected

When the comparison was repeated in a later campaign, *negative gaps* appeared: SENDA solutions apparently better than the optimum. Following the alarm of Section 2, the routes of those solutions were counted.

Table 10: Solutions with cost below the BKS in the project data. None is a legitimate record.

Instance	BKS (k)	Solution	Routes	Verdict
B-n51-k7	1032 (7)	1016	8	uses one vehicle too many
B-n57-k7	1153 (7)	1140	8	uses one vehicle too many
E-n30-k3	534 (3)	503	4	uses one vehicle too many
X-n247-k50, X-n420-k130	—	—	—	invalid, already excluded in v2

10.3 The cause

The program that launched SENDA in the experiments (the *runner*) told it “free fleet” even on A/B/E, and its verification checked coverage and capacity, but not the number of routes. HGS, on the other hand, was run with the fixed fleet, and FILO2’s fleet was verified. In total, 327 SENDA runs with more than k routes were accepted as valid. A later review of the whole project folder showed that the same problem affects the A/B/E sensitivity profiles (equalized CPU and parallel portfolio), whose mean SENDA gaps are also negative, and version 1 of the protocol.

10.4 The correction

1. The published analysis was repeated from the raw data and the same comparison table was obtained, byte for byte. This confirmed that the reanalysis introduced no changes of its own.
2. The same fleet rule was applied to all methods. In the original data, three instances are left without any valid SENDA replicate, so those data no longer support the comparison.
3. The experiment was repeated with the corrected runner (fleet equal to k), the same budget, the same seeds and HGS in the same block. With the fleet specified, SENDA respected it in all 630 runs: the extra vehicle came from the runner, not from the algorithm. That is the result of Table 7.

Methodological lesson: verify everyone equally

A comparison is fair only if *all* methods are verified with the same rule. Here, a difference in treatment —fixed fleet for HGS, free fleet for SENDA— turned three invalid solutions into “wins”. The warning signs were there: families B and E had negative mean gaps. They just had to be looked at with suspicion. The correction keeps the qualitative conclusion (SENDA is almost optimal on A/B/E), but changes the magnitude from 0.012 % to 0.111 % and removes every SENDA win.

To reproduce

Original data: canonical CSV `f3a1164c85f4....`. Rerun with a verified fixed fleet: contract `fef9dd7ee521....`. To repeat the verification with your own results, check in each solution the number of routes against the fleet of the instance, besides capacity and coverage (Section 13).

11 Discussion

Two regimes. On instances of up to one hundred customers, SENDA is within one tenth of a point of the BKS in two seconds; HGS is better, but only slightly. On instances of 199 to 428

customers, the gap to HGS and FILO2 grows to between 2 and 3 points. The cascade suggests why: on X, each arm has little more than one second, and in that time its ALNS barely improves what local search already achieved (4.971 % versus 5.244 % on the feasible solutions). Time, not construction, is the bottleneck.

Statistical significance versus practical relevance. A difference of 0.1 % in distance may be irrelevant or important depending on the operation. When routing is integrated with three-dimensional loading constraints, time windows and operational rules, those constraints usually weigh more than differences in pure routing. Quantifying it on the integrated problem is future work; this document is limited to the classic CVRP.

Limitations. (i) The main campaign and the fixed-fleet rerun were executed on different machines. Within each comparison all methods share a machine, but figures from different sources should not be mixed. (ii) The holdout contains only 10 instances from one family. (iii) On X, the cascade uses the A/B/E portfolio and not the X variant of the main campaign (4.150 % versus 4.119 %). (iv) The cascade was measured with the ablation binary, not the confirmatory one.

12 Conclusions

SENDA shows that a single-trajectory-per-run architecture, auditable and without a population, reaches almost optimal quality on small classic instances: a gap of 0.111 % in 2 s with a verified fleet, although HGS is equal or better on all of them. On X instances, HGS and FILO2 keep a clear advantage. The cascade and the ablation agree that the ALNS is the dominant mechanism, that moves between routes form a second level and that the selector and the penalized arm fine-tune the result.

Methodologically, the project leaves a lesson worth more than any figure: verification must be uniform, negative gaps are alarms and one’s own results must be audited too. Correcting them publicly does not weaken the work: it makes it credible.

Future work. (1) Give the ALNS more effective time on large instances, for example by keeping each arm from repeating construction and polishing. (2) Measure the impact of these differences on the integrated problem with three-dimensional loading. (3) Extend the holdout to other families and up to $n = 1000$.

To reproduce: artifacts

Campaign v2 (X, ablation, hold-out)	contract 36f29f1f1795..., CSV f3a1164c85f4...
Fleet correction	contract fef9dd7ee521...
Quality cascade	contract dd6b8817...; ablation binary 4002751d...
Evaluated core and HGS-CVRP	core binary d7921fe1...; HGS-CVRP binary 4e53fb18...
Figures in this document	data in paper/datos/ of this repository
Full artifacts	author’s archive; shared on request (contacto@lattimex.com)

13 Try it on your computer

The LATTIMEX routing engine uses a later version of SENDA (engine 1.6.0): it keeps its components and adds the parallel portfolio of Section 5.8. The figures in this guide were measured with binary d7921fe1..., not with the one in the repository. The engine is published under the MIT license at <https://github.com/Erick-MF28/lattimex>. In the repository the core is `native/send_core.cpp`; around it there is a pipeline that turns it into a real planner: effective capacity, three-dimensional placement of the boxes, fixed-fleet mediation and unloading plan by

door. The comparison programs (HGS-CVRP, FILO2 and the campaign scripts) are not part of the repository.

1. Install and check the engine. You need Python 3.10 or later and a C++ compiler (on Windows `pip install ziglang` is enough).

```
git clone https://github.com/Erick-MF28/lattimex.git
cd lattimex
pip install -r requirements.txt
python -m lattimex build          # compiles the core and the 3D packer
python -m lattimex probe          # deterministic fingerprint (see below)
python -m unittest discover -s tests/engine
```

The deterministic fingerprint summarizes a fixed run of 3000 iterations. It depends on the C++ standard library, because `std::shuffle` and the random distributions are implemented differently in each: 6457b2dd44660773 with zig/libc++ (the Windows installer) and 285bf1758e5dd6a1 with g++/libstdc++ (Linux and MinGW). If your build gives the fingerprint of its library, your binary computes exactly the same as the published one, even if the file is not identical byte for byte.

2. See it work on real streets. The engine builds its own map from OpenStreetMap and serves it to the graphical interface, the LATTIMEX Planner, which opens in the browser:

```
python -m lattimex map --bbox 20.47,-100.52,20.82,-100.22 --name queretaro
python -m lattimex serve          # then open https://lattimex.com/planner
```

In the Planner's *Guías* (shipments) section, the *Cargar ejemplo* button adds 300 synthetic deliveries in Querétaro, six vans and five drivers; *Optimizar* computes the routes and the placement of every box. The computations and the data stay on your computer. The Planner interface is currently in Spanish.

3. Repeat a comparison fairly. To measure the core on reference instances (CVRPLIB) against another solver, obtain that solver from its official source and follow the rules of Section 8:

1. the same wall clock for all methods, on the same machine and on a single thread;
2. at least 10 replicates with different seeds per instance;
3. the same fleet for everyone, fixed in advance;
4. an independent verifier that recomputes distance, capacity, coverage and number of routes;
5. paired comparisons per instance and correction for multiple comparisons (Holm).

A negative gap against the best known solution is almost always an alarm: check the fleet and the rounding of distances first.

Key idea

Open source lets anyone repeat the measurements. If your results differ from those in this guide, write to us: a well-documented discrepancy is a contribution.

References

- [1] Luca Accorsi and Daniele Vigo. Routing one million customers in a handful of minutes. *arXiv:2306.14205*, 2023.
- [2] Philippe Augerat, José Manuel Belenguer, Enrique Benavent, Ángel Corberán, Denis Naddef, and Giovanni Rinaldi. Computational results with a branch and cut code for the capacitated vehicle routing problem. Technical Report RR 949-M, Institut National Polytechnique de Grenoble, 1995.
- [3] Nicos Christofides and Samuel Eilon. An algorithm for the vehicle-dispatching problem. *Journal of the Operational Research Society*, 20(3):309–318, 1969.
- [4] CVRPLIB. CVRPLIB: Capacitated vehicle routing problem library. <http://vrp.atd-lab.inf.puc-rio.br/>, 2026. Accessed June 2026.
- [5] George B. Dantzig and John H. Ramser. The truck dispatching problem. *Management Science*, 6(1):80–91, 1959.
- [6] Bradley Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- [7] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [8] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [9] Jan Karel Lenstra and Alexander H. G. Rinnooy Kan. Complexity of vehicle routing and scheduling problems. *Networks*, 11(2):221–227, 1981.
- [10] Eduardo Queiroga, Ruslan Sadykov, Eduardo Uchoa, and Thibaut Vidal. 10,000 optimal CVRP solutions for testing machine learning based heuristics. *AAAI Workshop on Machine Learning for Operations Research*, 2021.
- [11] Stefan Ropke and David Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455–472, 2006.
- [12] Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming – CP98*, volume 1520 of *Lecture Notes in Computer Science*, pages 417–431. Springer, 1998.
- [13] Paolo Toth and Daniele Vigo. The granular tabu search and its application to the vehicle-routing problem. *INFORMS Journal on Computing*, 15(4):333–346, 2003.
- [14] Eduardo Uchoa, Diego Pecin, Artur Pessoa, Marcus Poggi, Thibaut Vidal, and Anand Subramanian. New benchmark instances for the capacitated vehicle routing problem. *European Journal of Operational Research*, 257(3):845–858, 2017.
- [15] Thibaut Vidal. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140:105643, 2022.
- [16] Thibaut Vidal, Teodor Gabriel Crainic, Michel Gendreau, Nadia Lahrichi, and Walter Rei. A hybrid genetic algorithm for multidepot and periodic vehicle routing problems. *Operations Research*, 60(3):611–624, 2012.
- [17] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

A Glossary

BKS Best known solution of a reference instance.

Gap Relative difference between the cost obtained and the BKS (equation 2).

Local optimum Solution that no move of a given neighborhood can improve.

Replicate Independent run with another seed. Here 10 per instance are used.

Arm One independent search within a trial; the portfolio has two or three.

Wall-clock time Real elapsed time, measured with a clock; distinct from CPU time.

Construction Procedure that generates the first complete solution.

Local search Repeated application of small changes that improve the solution.

Granular restriction Limiting moves to pairs of nearby customers.

ALNS Adaptive large neighborhood search: destroys and repairs parts of the solution with operators whose weights learn.

Simulated annealing Rule that accepts worse solutions with a probability that decreases over time.

Regret-2 Repair that first inserts the customer that would lose the most if left for later.

SWAP* Exchange of two customers between routes, each at its best position in the other route.

Portfolio SENDA configuration evaluated on A/B/E: base branch, FAST branch and penalized arm.

Penalized arm Search that tolerates temporary capacity excesses in exchange for a penalty.

Contract Frozen file with code, parameters, instances and tests, identified by its SHA-256 fingerprint.

Holdout Set of instances reserved and not observed during design.

Holm Correction for multiple comparisons that controls the family-wise error.

Big-O Notation that describes how the number of operations of an algorithm grows with the size of the input (Section 7).

B Literal rule of the FAST selector

Algorithm 2 Additional configuration chosen by FAST from $\phi(I)$

Input: n , k , load pressure ℓ , demand CV c_q , radial CV c_r , angular concentration α , density η

```
1: if  $n < 150$  then
2:   if  $n \geq 70$  then return granular_k25
3:   if  $60 \leq n < 70$  then
4:     if  $\eta \geq 0.0085$  or  $\alpha < 0.35$  then return large_destroy_k25
5:     if  $\alpha \geq 0.80$  then return long_10k_k25
6:     if  $k \geq 10$  then return granular_k40
7:     if  $c_q < 0.50$  then return long_10k_k25
8:     return baseline
9:   end if
10:  if  $45 \leq n \leq 65$  and  $\ell < 0.93$  and  $c_q < 0.50$  then return medium_destroy_k25
11:  return baseline
12: end if
13: if  $c_q \geq 0.50$  and ( $n \leq 245$  or  $k \geq 70$ ) then return large_destroy_k25
14: if  $k \leq 16$  and  $[(c_q < 0.30 \text{ and } c_r < 0.38) \text{ or } (c_q < 0.70 \text{ and } \alpha \geq 0.88)]$  then return
    granular_k25
15: if  $k \geq 55$  and  $c_q \geq 0.45$  and  $\alpha \geq 0.90$  then return granular_k25
16: if  $n \geq 300$  and  $c_q \geq 1.0$  then return baseline
17: return long_10k_k25
```

If the chosen configuration is **baseline**, the trial runs only the base branch and its penalized arm. Otherwise it runs the base branch, the chosen branch and the penalized arm of the latter.

C Errata with respect to version 2

- **A/B/E:** “SENDA 0.0120 % versus HGS 0.0059 %, 3/33/25, $p = 0.00406$ ” is replaced by “SENDA 0.111 % versus HGS 0.004 %, 0/37/23, $p = 2.4 \times 10^{-7}$ (60 pairs, verified fixed fleet)”.
- **Verification:** the statement that the fleet of every solution was verified was not true for SENDA in campaign v2; it was corrected in the rerun.
- **Gates:** with the verified fleet, the gate of the A/B/E comparison of campaign v2 fails; those of X, ablation and holdout pass unchanged.
- **Per-instance map figure:** the three cells favorable to SENDA corresponded to solutions with $k + 1$ vehicles (Figure 6).